

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\integration\SimpsonIntegrator.java

- Specification

module SimpsonIntegrator

type

```
relativeAccuracy: real          // relAcc
absoluteAccuracy: real         // absAcc
minimallterationCount: int     // minlter
maximallterationCount: int     // maxlter
min: real
max: real
maxEval: int
evaluations_used: int
iterations_used: int
converged_by_value_accuracy: bool
converged_by_interval_shrinkage: bool
not_converged_by_accuracy: bool
ex: set of {
    None,
    NullArgumentException,
    NotStrictlyPositiveException,
    NumberIsTooSmallException,
    NumberIsTooLargeException,
    NotANumberException,
    NotFiniteNumberException,
    TooManyEvaluationsException,
    MaxCountExceededException
}
```

var

```
MIN_SIMPSON_ITER = 3
MAX_SIMPSON_ITER = 30
```

process integrate(f, min, max, maxEval,
 relativeAccuracy, absoluteAccuracy,
 minimallterationCount, maximallterationCount: real)
 result: real, ex: set of { ... }

Functional Scenario Form

```
f = 0                                && ex = NullArgumentException
||
maxEval <= 0                        && ex =
```

```

NotStrictlyPositiveException ||

    relativeAccuracy <= 0    && ex = NotStrictlyPositiveException ||

    minIterationCount < MIN_SIMPSON_ITER                && ex =
NumberIsTooSmallException ||
    maxIterationCount > MAX_SIMPSON_ITER                && ex =
NumberIsTooLargeException ||
    minIterationCount >= maxIterationCount              && ex =
NumberIsTooSmallException ||
    isNaN=1                && ex = NotANumberException ||
    isInfinite=1           && ex = NotFiniteNumberException ||
    min >= max              && ex =
NumberIsTooLargeException ||
    evaluations_used > maxEval    && ex = TooManyEvaluationsException ||
        iterations_used >= maxIterationCount            && ex =
MaxCountExceededException ||
    evaluations_used <= maxEval

&& ex = None

```

end_process

end_module

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;

```

```

import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeException;
import org.apache.commons.math4.legacy.exception.NotANumberException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberException;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsException;
import org.apache.commons.math4.legacy.exception.MaxCountExceededException;
/**
 * Implements <a href="http://mathworld.wolfram.com/SimpsonsRule.html">
 * Simpson's Rule</a> for integration of real univariate functions.
 *
 * See <b>Introduction to Numerical Analysis</b>, ISBN 038795452X, chapter 3.
 *
 * <p>
 * This implementation employs the basic trapezoid rule to calculate Simpson's
 * rule.
 *
 * <p>
 * <em>Caveat:</em> At each iteration, the algorithm refines the estimation by
 * evaluating the function twice as many times as in the previous iteration;
 * When specifying a {@link #integrate(int,UnivariateFunction,double,double)
 * maximum number of function evaluations}, the caller must ensure that it
 * is compatible with the {@link #SimpsonIntegrator(int,int) requested minimal
 * number of iterations}.
 *
 * @since 1.2
 */

```

```

public class SimpsonIntegrator extends BaseAbstractUnivariateIntegrator
{
    /** Maximal number of iterations for Simpson. */
    private static final int SIMPSON_MAX_ITERATIONS_COUNT = 30;
    /**
     * Build a Simpson integrator with given accuracies and iterations
counts.
     * @param relativeAccuracy relative accuracy of the result
     * @param absoluteAccuracy absolute accuracy of the result
     * @param minimalIterationCount Minimum number of iterations.
     * @param maximalIterationCount Maximum number of iterations.
     * It must be less than or equal to 30.
     * @throws org.apache.commons.math4.legacy.exception.NotStrictlyPos
itiveException
     * if {@code minimalIterationCount <= 0}.
     * @throws org.apache.commons.math4.legacy.exception.NumberIsTooSma
llException
     * if {@code maximalIterationCount < minimalIterationCount}.
     * is lesser than or equal to the minimal number of iterations
     * @throws NumberIsTooLargeException if {@code maximalIterationCount
> 30}.
     */
    public SimpsonIntegrator(final double relativeAccuracy,
                             final double absoluteAccuracy,
                             final int minimalIterationCount,
                             final int maximalIterationCount) {
        super(relativeAccuracy, absoluteAccuracy, minimalIterationCount
, maximalIterationCount);
        if (maximalIterationCount > SIMPSON_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
                                                SIMPSON_MAX_ITERATIONS_
COUNT, false);
        }
    }
    /**
     * Build a Simpson integrator with given iteration counts.
     * @param minimalIterationCount Minimum number of iterations.
     * @param maximalIterationCount Maximum number of iterations.
     * It must be less than or equal to 30.
     * @throws org.apache.commons.math4.legacy.exception.NotStrictlyPos
itiveException
     * if {@code minimalIterationCount <= 0}.
     * @throws org.apache.commons.math4.legacy.exception.NumberIsTooSma
llException

```

```

    * if {@code maximalIterationCount < minimalIterationCount}.
    * is lesser than or equal to the minimal number of iterations
    * @throws NumberIsTooLargeException if {@code maximalIterationCount > 30}.
    */
    public SimpsonIntegrator(final int minimalIterationCount,
                             final int maximalIterationCount) {
        super(minimalIterationCount, maximalIterationCount);
        if (maximalIterationCount > SIMPSON_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
                                                SIMPSON_MAX_ITERATIONS_
COUNT, false);
        }
    }
    /**
     * Construct an integrator with default settings.
     */
    public SimpsonIntegrator() {
        super(DEFAULT_MIN_ITERATIONS_COUNT, SIMPSON_MAX_ITERATIONS_COUNT);
    }
    /** {@inheritDoc} */
    @Override
    protected double doIntegrate() {
        // Simpson's rule requires at least two trapezoid stages.
        // So we set the first sum using two trapezoid stages.
        final TrapezoidIntegrator qtrap = new TrapezoidIntegrator();
        final double s0 = qtrap.stage(this, 0);
        double oldt = qtrap.stage(this, 1);
        double olds = (4 * oldt - s0) / 3.0;
        while (true) {
            // The first iteration is the first refinement of the sum.
            iterations.increment();
            final int i = getIterations();
            final double t = qtrap.stage(this, i + 1); // 1-
stage ahead of the iteration
            final double s = (4 * t - oldt) / 3.0;
            if (i >= getMinimalIterationCount()) {
                final double delta = JdkMath.abs(s - olds);
                final double rLimit = getRelativeAccuracy() * (JdkMath.
abs(olds) + JdkMath.abs(s)) * 0.5;
                if (delta <= rLimit ||
                    delta <= getAbsoluteAccuracy()) {
                    return s;
                }
            }
        }
    }

```

```

        }
    }
    olds = s;
    oldt = t;
}
}
}
//vt
class SIVT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n=== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ----- sample functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_a^b 5 \, dx = 5(b-a)$ 
    static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_0^4 (x-2) \, dx = 0$ 
    static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^\pi \sin = 2$ 
    public void Vtest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 3;
        final int MAX_ITER = 12;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));
    }
}

```

```

        SimpsonIntegrator S = new SimpsonIntegrator(REL, ABS, MIN_ITER,
MAX_ITER);
        // VT01: f == null -> NullArgumentException
        expectThrow("VT01 f=null",
            () -> S.integrate(1000, null, 0.0, 1.0),
            NullArgumentException.class);
        // VT02: maxIter > 30 (ctor) -> NumberIsTooLargeException
        expectThrow("VT02 maxIter>30 (ctor)",
            () -> new SimpsonIntegrator(1e-8, 1e-8, 3, 64),
            NumberIsTooLargeException.class);
        // VT03: minIter == maxIter (ctor) -> NumberIsTooSmallException
        expectThrow("VT03 minIter==maxIter (ctor)",
            () -> new SimpsonIntegrator(1e-8, 1e-8, 5, 5),
            NumberIsTooSmallException.class);
        // VT04: Infinite endpoints -> NotFiniteNumberException
        expectThrow("VT04 -Infinity at min",
            () -> S.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY
, 1.0),
            NotFiniteNumberException.class);
        // VT05: min ≥ max (beta1 strict) -> NumberIsTooLargeException
        expectThrow("VT05 min>max",
            () -> S.integrate(1000, F_CONST, 1.0, -1.0),
            NumberIsTooLargeException.class);
        // VT06:
        try {
            SimpsonIntegrator S_tight = new SimpsonIntegrator(1e-
30, 1e-30, 3, 4);
            expectThrow("VT6 hit max iterations (tight accuracy, narrow
band)",
                () -> S_tight.integrate(1 << 26, F_SIN, 0.0, 100.0)
,
                MaxCountExceededException.class);
        } catch (Throwable t) {
            System.out.println("VT11 note: " + t.getClass().getSimpleNa
me() + " - " + t.getMessage());
        }
        // VT07: ZERO
        headline("VT07 ZERO small interval within budget");
        double r12 = S.integrate(BIG_BUDGET, F_ZERO, -1e-8, 1e-8);
        System.out.printf("VT12 result=%.17g%n", r12);
        // VT08: CONST
        headline("VT08 ctor(minIter,maxIter) boundary success (3,30)");
        SimpsonIntegrator S_mOnly_ok = new SimpsonIntegrator(3, 30);

```

```

        double vt23 = S_mOnly_ok.integrate(1 << 22, F_CONST, 0.0, 1e-
4);
        System.out.printf("VT23 result=%.17g (expected≈%g)%n", vt23, 5.
0 * 1e-4);
        // VT09:
        expectThrow("VT09 ctor(minIter,maxIter) maxIter>30",
            () -> new SimpsonIntegrator(3, 31),
            NumberIsTooLargeException.class);

        // VT10:
        headline("VT10 LIN [0,4]");
        double r15 = S.integrate(Math.max(BIG_BUDGET, 1 << 21), F_LIN,
0.0, 4.0);
        System.out.printf("VT15 result=%.17g |err|=%g%n", r15, Math.abs
(r15 - 0.0));
        // VT11:
        headline("VT11 wide finite SIN sanity [-100,100]");
        double r19 = S.integrate(Math.max(BIG_BUDGET, 1 << 23), F_SIN,
-100.0, 100.0);
        System.out.printf("VT19 result=%.17g%n", r19);
        System.out.println("\n[Done] SimpsonIntegrator V-
method cases executed.");
    }
}
//ft
class SIFT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n=== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> e
xpected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s%n", name, expe
cted.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " -
" + t.getMessage());
            System.out.printf("[%s] %s threw expected%n",
                t.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
}

```

```

    }
    // ----- sample functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0;           //  $\int_a$ 
^b 5 dx = 5(b-a)
    static final UnivariateFunction F_LIN = x -> x - 2.0;         //  $\int_0$ 
^4 (x-2) dx = 0
    static final UnivariateFunction F_SIN = Math::sin;           //  $\int_0$ 
^ $\pi$  sin = 2
    public void Ftest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 3;
        final int MAX_ITER = 12;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));

        SimpsonIntegrator S = new SimpsonIntegrator(REL, ABS, MIN_ITER,
MAX_ITER);
        // -----
- Random pack: FT01 - FT11 (SimpsonIntegrator) -----
        // FT01: reversed interval with huge magnitudes -> NumberIsTooLarg
eException
        expectThrow("FT01 reversed extreme interval",
            () -> S.integrate(1000, F_SIN,
                5.005341669616643E307, -1.1209145429141500E308),
            NumberIsTooLargeException.class);
        // FT02: degenerate interval at extreme negative -> NumberIsTooLar
geException
        expectThrow("FT02 degenerate interval (min==max, extreme)",
            () -> S.integrate(1000, F_CONST,
                -1.7976931348623157E308, -1.7976931348623157E308),
            NumberIsTooLargeException.class);
        // FT03: NaN at min -> NotANumberException
        expectThrow("FT03 NaN min",
            () -> S.integrate(1000, F_LIN, Double.NaN, 1.0),
            NotANumberException.class);
        // FT04: NaN at max -> NotANumberException
        expectThrow("FT04 NaN max",
            () -> S.integrate(1000, F_SIN, 0.0, Double.NaN),
            NotANumberException.class);
        // FT05: +Infinity at max -> NotFiniteNumberException
        expectThrow("FT05 +Infinity at max",
            () -> S.integrate(1000, F_SIN, -
10.0, Double.POSITIVE_INFINITY),

```

```

        NotFiniteNumberException.class);
    // FT06: -Infinity at min -> NotFiniteNumberException
    expectThrow("FT06 -Infinity at min",
        () -> S.integrate(1000, F_CONST, Double.NEGATIVE_INFINITY,
10.0),
        NotFiniteNumberException.class);
    // FT07: tiny budget on very wide interval -> TooManyEvaluationsEx
ception
    expectThrow("FT07 budget=1 on wide interval",
        () -> S.integrate(1, F_SIN, -1.0e3, 1.0e3),
        TooManyEvaluationsException.class);
    // FT08: ZERO on tiny symmetric interval (success; integral=0)
    headline("FT08 ZERO tiny symmetric interval");
    double ft08 = S.integrate(BIG_BUDGET, F_ZERO, -1e-12, 1e-12);
    System.out.printf("FT08 result=%.17g%n", ft08);
    // FT09: CONST near-degenerate safe epsilon (success)
    headline("FT09 CONST near-degenerate safe");
    double a09 = 3.141592653589793;
    double eps09 = Math.max(1e-9, 16 * Math.ulp(a09));
    double ft09 = S.integrate(BIG_BUDGET, F_CONST, a09, a09 + eps09);
    System.out.printf("FT09 result=%.17g (expected=%g, eps=%g)%n", ft0
9, 5.0 * eps09, eps09);
    // FT10: SIN on random moderate interval (success; boosted budget)
    headline("FT10 SIN moderate random window");
    int bud10 = Math.max(BIG_BUDGET, 1 << Math.min(28, (MAX_ITER + 12)
));
    double ft10 = S.integrate(bud10, F_SIN, 1.23456789012345, 23.45678
90123456);
    System.out.printf("FT10 result=%.17g (budget=%d)%n", ft10, bud10);
    // FT11: LIN on wide finite random range (success; boosted budget)
    headline("FT11 LIN wide finite random");
    int bud11 = Math.max(BIG_BUDGET, 1 << Math.min(28, (MAX_ITER + 12)
));
    double ft11 = S.integrate(bud11, F_LIN, -
5.43210987654321E2, 7.65432109876543E2);
    System.out.printf("FT11 result=%.17g (budget=%d)%n", ft11, bud11);
    System.out.println("\n[Done] SimpsonIntegrator FUZZING cases ex
ecuted.");
}
}
//Z3
class SIZ3 {
    // ---- helpers ----
    private static void headline(String name) {

```

```

        System.out.println("\n==== " + name + " =====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
}
// ---- simple functions ----
static final UnivariateFunction F_ZERO = x -> 0.0;
static final UnivariateFunction F_CONST = x -> 5.0;
static final UnivariateFunction F_SIN = Math::sin;
static final UnivariateFunction F_LIN = x -> x - 2.0;
public void Z3test() {
    final SimpsonIntegrator S_OK = new SimpsonIntegrator(1e-3, 1e-3, 3, 4);
    // Z301 -----
    expectThrow("Z301 f=null",
        () -> S_OK.integrate(10, null, 0.0, 1.0),
        NullPointerException.class);
    // Z302 -----
    SimpsonIntegrator S302 = new SimpsonIntegrator(0.0, 0.5, 3, 4);
    expectThrow("Z302 maxEval<=0",
        () -> S302.integrate(0, F_ZERO, 0.0, 1.0),
        NotStrictlyPositiveException.class);
    // Z303 -----
    expectThrow("Z303 rel<=0 AND abs<=0 (ctor)",
        () -> new SimpsonIntegrator(0.0, 0.0, 3, 4),
        NotStrictlyPositiveException.class);
    // Z304 -----
    expectThrow("Z304 minIter<3 (ctor)",

```

```

        () -> new SimpsonIntegrator(0.5, 0.0, 0, 1),
        NumberIsTooSmallException.class);
// Z305 -----
-----
expectThrow("Z305 maxIter>30 (ctor)",
    () -> new SimpsonIntegrator(0.0, 0.5, 3, 31),
    NumberIsTooLargeException.class);
// Z306 -----
-----
expectThrow("Z306 minIter>=maxIter (ctor)",
    () -> new SimpsonIntegrator(0.0, 0.5, 3, 0),
    NumberIsTooSmallException.class);
// Z307 -----
-----
SimpsonIntegrator S307 = new SimpsonIntegrator(0.0, 0.5, 3, 4);
expectThrow("Z307 NaN at min",
    () -> S307.integrate(10, F_LIN, Double.NaN, 1.0),
    NotANumberException.class);
// Z308 -----
-----
SimpsonIntegrator S308 = new SimpsonIntegrator(0.5, 0.0, 3, 4);
expectThrow("Z308 min==max",
    () -> S308.integrate(10, F_CONST, 0.0, 0.0),
    NumberIsTooLargeException.class);
// Z309 -----
-----
SimpsonIntegrator S309 = new SimpsonIntegrator(0.0, 0.5, 3, 4);
expectThrow("Z309 budget exhausted (maxEval=1 surrogate)",
    () -> S309.integrate(1, F_SIN, 0.0, 1.0),
    TooManyEvaluationsException.class);
// Z310 -----
-----
try {
    SimpsonIntegrator S310 = new SimpsonIntegrator(1e-30, 1e-
30, 3, 4);
    expectThrow("Z310 hit max iterations (tight acc, small band
)",
        () -> S310.integrate(1 << 24, F_SIN, 0.0, 100.0),
        MaxCountExceededException.class);
} catch (Throwable t) {
    System.out.println("Z310 note: " + t.getClass().getSimpleName() + " - " + t.getMessage());
}

```

```

// Z311 -----
-----
    headline("Z311 success (converged)");
    SimpsonIntegrator S311 = new SimpsonIntegrator(0.5, 0.0, 3, 4);
    double z311 = S311.integrate(1000, F_CONST, 0.0, 1e-4);
    System.out.printf("Z311 result=%.17g (expected=%g)%n", z311, 5.
0 * 1e-4);
    System.out.println("\n[Done] Z301-Z311 executed.");
}
}
class test {
    public static void main(String[] args) {
        // VT
        SIVT t1 = new SIVT();
        t1.Vtest();
        // FT
        SIFT t2 = new SIFT();
        // t2.Ftest();
        // // Z3
        SIZ3 t3 = new SIZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 11
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SimpsonIntegrator.java	75.0 %	12	4	16
> SIFT	0.0 %	0	2	2
> SIZ3	0.0 %	0	2	2
▼ SimpsonIntegrator	100.0 %	10	0	10
SimpsonIntegrator(double, double, i	100.0 %	2	0	2
SimpsonIntegrator(int, int)	100.0 %	2	0	2
doIntegrate()	100.0 %	6	0	6
> SIVT	100.0 %	2	0	2
> test		0	0	0

3. V-Method testing instruction coverage: 95.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SimpsonIntegrator.java	35.7 %	406	731	1,137
> SIFT	3.5 %	12	327	339
> SIZ3	3.7 %	12	311	323
> SIVT	74.8 %	252	85	337
> test	83.3 %	15	3	18
▼ SimpsonIntegrator	95.8 %	115	5	120
SimpsonIntegrator(double, double, i	100.0 %	19	0	19
SimpsonIntegrator(int, int)	100.0 %	17	0	17
doIntegrate()	100.0 %	79	0	79

4. Fuzz testing branch coverage: 60%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SimpsonIntegrator.java	50.0 %	8	8	16
> SIVT	0.0 %	0	2	2
> SIZ3	0.0 %	0	2	2
▼ SimpsonIntegrator	60.0 %	6	4	10
SimpsonIntegrator(double, double, i	50.0 %	1	1	2
doIntegrate()	83.3 %	5	1	6
> SIFT	100.0 %	2	0	2
> test		0	0	0

5. Fuzz testing instruction coverage: 74.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SimpsonIntegrator.java	36.1 %	410	727	1,137
> SIVT	3.6 %	12	325	337
> SIZ3	3.7 %	12	311	323
▼ SimpsonIntegrator	74.2 %	89	31	120
SimpsonIntegrator(double, double, i	52.6 %	10	9	19
doIntegrate()	100.0 %	79	0	79
> SIFT	83.2 %	282	57	339
> test	83.3 %	15	3	18

6. Z3 branch coverage: 70%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SimpsonIntegrator.java	56.2 %	9	7	16
> SIFT	0.0 %	0	2	2
> SIVT	0.0 %	0	2	2
▼ SimpsonIntegrator	70.0 %	7	3	10
doIntegrate()	83.3 %	5	1	6
SimpsonIntegrator(double, double, i	100.0 %	2	0	2
> SIZ3	100.0 %	2	0	2
> test		0	0	0

7. Z3 instruction coverage: 81.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SimpsonIntegrator.java	33.0 %	375	762	1,137
> SIFT	3.5 %	12	327	339
> SIVT	3.6 %	12	325	337
> SIZ3	73.7 %	238	85	323
▼ SimpsonIntegrator	81.7 %	98	22	120
SimpsonIntegrator(double, double, i	100.0 %	19	0	19
doIntegrate()	100.0 %	79	0	79
> test	83.3 %	15	3	18